

Problème 1 : factorisation en nombres premiers

Observations concernant l'algorithme. En étudiant l'algorithme on se rend compte que i est incrémenté à chaque itération de la boucle externe. La boucle interne n'est appliquée que si x est divisible par i . Celle-ci modifie la valeur de x (mais pas de i) en la divisant par i . La boucle externe s'arrête quand x est égal à 1, ce qui peut arriver de deux manières. Soit x est un nombre premier et la boucle interne n'est donc jamais appliquée avant que x ne soit égal à i , ce qui constituera alors la fin de l'algorithme puisque x deviendra égal à 1 via la division de x par i . Soit x n'est pas premier, et sa valeur sera donc diminuée via la division dès qu'un diviseur est trouvé, ce qui va nous rapprocher plus rapidement du moment où x sera égal à i , et donc ensuite à 1 après la division, ce qui terminera également l'algorithme.

- a) Parmi les trois propositions, seule la dernière était effectivement un invariant de boucle.
- (1) (2 points) *Faux*. Un nombre n est ajouté dans r que dans le cas où $x \% i == 0$, c'est à dire si x divise i . Pour un nombre x donné et un nombre premier $p < i$, il se peut très bien que p ne divise pas x . Par exemple, si x a une valeur 7, cet algorithme ne va pas stocker 5 dans r , même si i sera plus grand que 5 à un moment donné.
 - (2) (2 points) *Faux*. Seuls les diviseurs premiers sont stockés. En effet, supposons qu'un nombre soit divisible par un nombre non premier q , l'algorithme aura alors déjà divisé x par tous les facteurs premiers de q (puisque i est incrémenté à la fin de chaque itération, en partant de 2 ; et que les facteurs premiers de q sont $< q$ puisqu'il n'est pas premier) et le nouveau x ne sera plus divisible par q .
 - (3) (3 points) *Vrai*. Avant d'entrer dans la boucle, r est vide et i vaut 2. L'assertion est donc vraie puisqu'il n'existe pas de nombre premier < 2 . Supposons que cela soit vrai pour $i < k$ (hypothèse d'induction), est-ce toujours vrai quand i atteindra la valeur k à la fin d'une itération ? Par induction, on sait que quand i vaut $k-1$ à la fin d'une itération complète, tous les diviseurs premiers de x strictement inférieurs à $k-1$ sont dans r . Or, à l'itération suivante on teste si i est divisible par $k-1$ (car i n'est incrémenté qu'après la boucle interne). Si $k-1$ n'est pas premier, il ne sera plus diviseur de x (cf. point (2)), et donc $k-1$ ne sera ajouté à r que s'il est à la fois un diviseur de x et un nombre premier. L'assertion est donc toujours vraie à la fin de l'exécution complète de cette itération.
- b) (4 points) Comme observé ci-dessus, la boucle interne permet d'accélérer la fin de l'algorithme quand on trouve un diviseur premier de x . Dans le pire des cas (cela arrive uniquement quand x est premier), la boucle interne n'est exécutée que quand i vaut x et ne fait qu'une seule itération. La complexité de cet algorithme est donc en $O(x)$ puisque dans ce cas, la boucle externe aura été exécutée pour i allant de 2 à x et que toutes les instructions du corps de la boucle sont en $O(1)$. Une erreur fréquente a été de dire que l'algorithme était en $O(k)$ (ce qui reviendrait à du $O(1)$ pour un nombre premier). Une erreur plus grave a été de dire que cet algorithme avait une complexité quadratique car il contient deux boucles imbriquées. Des réponses comme $O(x^2)$ ou $O(k^2)$ sont donc fausses au vu des arguments donnés ci-dessus.

Problème 2 : insertion dichotomique

Voici une solution pour `getIndex` et la pondération associée (6 points au total).

```
def getIndex(myList, elem):
    if (len(myList) == 1): # 1 point
        if elem < myList[0]:
            return 0
        else:
            return 1
    else:
        middle = len(myList) / 2 # 1 point
        if elem < myList[middle]:
            return getIndex(myList[0:middle], elem) # 2 points
        else:
            return middle + getIndex(myList[middle:], elem) # 2 points
```

Pour `insert` (4 points) il fallait utiliser des méthodes de liste : `insert`, `append`. La solution la plus facile est la suivante.

```
def insert(myList, indice, elem):
    myList.insert(indice, elem)
```

Une solution alternative est présentée ci-dessous.

```
def insert(myList, indice, elem):
    myList.append(0) # increase at the end
    #made the swap
    i = len(myList) - 1
    while i > indice:
        myList[i] = myList[i-1]
        i -= 1
    myList[indice] = elem
```

La "solution" suivante vaut 3 points sur 4 car cette solution ne modifie pas la variable la liste passée en paramètre, comme demandé dans l'énoncé. En effet, l'instruction `myList = partA + partB` crée une nouvelle liste et copie sa référence dans une variable `myList` **locale** à la fonction mais ne mets pas à jour la variable passée en paramètre (cf. explications dans le chapitre sur les listes).

```
def insert(myList, indice, elem):
    partA = myList[:indice]
    partB = myList[indice:]
    partA.append(elem)
    myList = partA + partB
```